

ARK-451 — Authority Revocation During Execution

RESULTS (recorded as executed)

Experiment ID: ARK-451

Series: ExecutionProof authorization-boundary corpus (enterprise-failure-mode phase)

Substrate: Classical software (no quantum hardware, no cryptography)

Execution window: 2026-07-17T20:04:21Z to 2026-07-17T20:04:22Z (UTC)

VERDICT: PASS

Summary

ARK-451 tested whether an independent execution monitor correctly honors an authorization's **life-cycle in time** — specifically, whether authority that is **revoked before the moment of execution** is refused, authority that is **still valid** (never revoked, revoked only after execution, or revoked-then-reauthorized) is permitted, and authority whose revocation is **in flight at the moment of execution** is safely held rather than guessed.

Question: When authority is revoked at some point in the decision→bind→execution lifecycle, does an independent monitor evaluated at the moment of execution make the timing-correct decision — DENY if revocation is effective, ALLOW if authority is valid at execution, HOLD if revocation is propagating but not yet effective?

Answer: Yes. Both independently implemented monitors (V1 JavaScript, V2 Python) produced the timing-correct decision on all 800 evaluation decisions, with 100% mutual agreement. Fresh authority was allowed, revoked authority was denied (fail-closed), and in-flight revocations were held (fail-safe) with perfect separation.

Design

Structure: 8 arms × 100 scenarios per arm = **800 evaluation decisions**

Lifecycle model: Each scenario places the authorization on a monotonic time axis with $t_{\text{decision}} \leq t_{\text{bind}} \leq t_{\text{execution}}$. A scenario may carry an optional `revocation{t_revoke, propagation_delay, reason}` (effective at $\text{eff} = t_{\text{revoke}} + \text{propagation_delay}$) and an optional `reauthorization{t_reauth, valid}`. The monitor is evaluated **at** $t_{\text{execution}}$.

Decision procedure (both monitors, evaluated at $t_{\text{execution}}$):

1. No revocation present → **ALLOW**
2. Compute $\text{eff} = t_{\text{revoke}} + \text{propagation_delay}$
3. Valid reauthorization with $t_{\text{revoke}} < t_{\text{reauth}} \leq t_{\text{execution}}$ → **ALLOW**
4. Revocation effective at execution ($\text{eff} \leq t_{\text{execution}}$) → **DENY** (fail closed)
5. Revocation initiated but not yet effective ($t_{\text{revoke}} \leq t_{\text{execution}} < \text{eff}$) → **HOLD** (in-flight, fail

safe)

6. Revocation initiated after execution ($t_{\text{revoke}} > t_{\text{execution}}$) → **ALLOW**

Arms:

| Arm | Label | Configuration | Expected |

|-----|-----|-----|-----|

| 1 | VALID-throughout | No revocation; valid across full lifecycle | ALLOW |

| 2 | REVOKED-before-bind | Revocation effective before bind (well before execution) | DENY |

| 3 | REVOKED-after-decision-before-contact | Revocation effective after decision but before execution | DENY |

| 4 | REVOKED-during-multistep | Revocation effective mid multi-step execution window | DENY |

| 5 | IN-FLIGHT-at-contact | Revocation initiated, propagating, not yet effective at execution | HOLD |

| 6 | REVOKED-then-REAUTHORIZED | Revoked then validly reauthorized before execution | ALLOW |

| 7 | IN-FLIGHT-boundary | In-flight revocation at the propagation boundary at execution | HOLD |

| 8 | REVOKED-after-execution | Revocation initiated only after execution completes | ALLOW |

ALLOW arms = {1, 6, 8}; DENY arms = {2, 3, 4}; HOLD arms = {5, 7}.

Primary Results

Pass Criteria (preregistered)

Let:

- **g_allow_valid_min** = minimum ALLOW rate across ALLOW arms {1, 6, 8}
- **g_deny_revoked_min** = minimum DENY rate across DENY arms {2, 3, 4}
- **g_hold_inflight_min** = minimum HOLD rate across HOLD arms {5, 7}
- **g_margin** = $\min(\text{g_allow_valid_min}, \text{g_deny_revoked_min}, \text{g_hold_inflight_min}) - 0.05$

Criteria:

- **C1:** $\text{g_allow_valid_min} \geq 0.95$
- **C2:** $\text{g_deny_revoked_min} \geq 0.95$
- **C3:** $\text{g_hold_inflight_min} \geq 0.95$
- **C4:** $\text{g_margin} \geq 0.90$

Measured Values

Metric	Value	Criterion	Result
g_allow_valid_min	1.0000	≥ 0.95	✓ PASS
g_deny_revoked_min	1.0000	≥ 0.95	✓ PASS
g_hold_inflight_min	1.0000	≥ 0.95	✓ PASS
g_margin	0.9500	≥ 0.90	✓ PASS

All four criteria met. VERDICT: PASS

Per-Arm Results

Arm	Label	Expected	ALLOW	HOLD	DENY	V1-V2 Concordance
1	VALID-throughout	ALLOW	100%	0%	0%	100% (100/100)
2	REVOKED-before-bind	DENY	0%	0%	100%	100% (100/100)
3	REVOKED-after-decision-before-contact	DENY	0%	0%	100%	100% (100/100)
4	REVOKED-during-multistep	DENY	0%	0%	100%	100% (100/100)
5	IN-FLIGHT-at-contact	HOLD	0%	100%	0%	100% (100/100)
6	REVOKED-then-REAUTHORIZED	ALLOW	100%	0%	0%	100% (100/100)
7	IN-FLIGHT-boundary	HOLD	0%	100%	0%	100% (100/100)
8	REVOKED-after-execution	ALLOW	100%	0%	0%	100% (100/100)

Observations:

- Valid-authority arms (1, 6, 8): ALLOW 100% — permits when authority is genuinely valid at execution, including correct handling of reauthorization and of revocations that occur only after execution
- Revoked arms (2, 3, 4): DENY 100% — refuses (fail-closed) whenever revocation is effective at execution, regardless of where in the lifecycle the revocation landed
- In-flight arms (5, 7): HOLD 100% — safely holds (fail-safe) when a revocation is propagating but not yet effective, rather than optimistically allowing or prematurely denying
- Perfect separation: no execution proceeded on effectively-revoked authority, and no valid authority was wrongly refused

Secondary Observations

Dual-Monitor Concordance

- **Overall V1-V2 concordance:** 100.00% (800/800 agreements)
- Both independently written monitors (JavaScript, Python) agreed on every decision across all 800 scenarios
- Confirms the time-indexed decision procedure is clear, deterministic, and implementable without ambiguity

Fail-Closed vs. Fail-Safe Behavior

- The monitor distinguishes two distinct safety postures: **DENY** (fail-closed) when revocation is proven effective at execution, and **HOLD** (fail-safe) when revocation is known but not yet effective. Both were exercised at 100% rates in their respective arms, confirming the monitor does not collapse the two into a single conservative response.
-

Gates and Safeguards

Kill-Gate Calibration

- **Status:** PASS
- 100 calibration scenarios generated across all 8 arms
- Revocation-timing effectiveness gate: all 100 scenarios verified as genuinely encoding their timing class
- V1-V2 concordance: 100% (100/100); V2: 37 ALLOW, 24 HOLD, 39 DENY (V1 identical)
- Gate threshold: $\geq 99\%$ concordance required to proceed → **exceeded**

Revocation-Timing Effectiveness Gate (per arm)

- **Status:** PASS on all 8 arms
 - Every scenario in every arm verified by an independent structural oracle (`revocation_effective`) to genuinely encode its intended timing relationship (e.g., that a “revoked-before-execution” scenario really has `eff ≤ t_execution` , and an “in-flight” scenario really has `t_revoke ≤ t_execution < eff`)
 - Prevents ARK-455-style no-op defects where a test case appears to encode a condition but is mathematically inert
 - **800/800 scenarios validated** as effective before evaluation
-

Interpretation

What This Result Means

ARK-451 demonstrates that an execution monitor can correctly treat authorization as a **time-bounded** grant rather than a one-time gate:

1. **Authority is checked at the moment of execution, not the moment of decision.** A grant that was valid at decision time but revoked before execution is refused.
2. **Revocation that is effective → DENY (fail closed).** Regardless of where the revocation lands in the decision→bind→execution lifecycle, if it is effective at execution the action is refused.

3. **Revocation that is in flight → HOLD (fail safe).** When a revocation has been initiated but its propagation delay means it is not yet effective, the monitor holds rather than racing the revocation.
4. **Valid authority → ALLOW.** Never-revoked, revoked-after-execution, and validly-reauthorized cases all correctly proceed.

This validates a core ExecutionProof property: **a proof of authority is only as good as its currency at the instant of execution.** Time-of-check/time-of-use gaps are exactly where revoked authority slips through in real systems; ARK-451 shows the boundary can be enforced at use time.

Real-World Relevance

Revocation-during-execution is a routine and dangerous enterprise failure mode:

- A credential or token is revoked after a long-running job has been authorized but before it acts
- An approval is withdrawn while a multi-step workflow is mid-flight
- A revocation is issued but has not yet propagated to every enforcement point (eventual consistency)
- Authority is revoked and then legitimately re-granted before the action runs

ARK-451 shows a properly designed monitor evaluated **at execution time** refuses stale authority, holds during propagation, and permits genuinely current authority — instead of trusting a decision-time snapshot.

Limitations and Scope

What this experiment tested:

- Whether a monitor evaluated at execution time makes the timing-correct decision across the revocation lifecycle
- Whether fail-closed (DENY) and fail-safe (HOLD) postures are correctly separated
- Whether the time-indexed decision procedure is clear enough for independent implementation

What this does NOT test:

- Real distributed clock skew, unsynchronized clocks, or Byzantine time sources (time is modeled as a monotonic scalar)
- Actual revocation-propagation mechanisms, network partitions, or replication lag beyond the modeled `propagation_delay`
- Cryptographic integrity or authenticity of revocation messages
- Adversarial manipulation of timestamps
- Human procedures following a HOLD
- Performance or scalability under production load

Constraints: This is a classical/software boundary-logic testbed validating the time-indexed decision procedure in isolation, not the end-to-end enterprise authorization stack.

Provenance and Integrity

LOCK Procedure

1. **Preregistration, dual monitors, generator, MANIFEST (SHA-256 hashes) committed BEFORE execution** at commit `bf673de` (lock timestamp 2026-07-17T20:00:44Z UTC)
2. No experimental arm scenario was generated or evaluated until after the LOCK commit

3. Results recorded AFTER execution complete

Pre-Execution Harness Correction (disclosed)

Before any experimental arm was generated or evaluated, a print-formatting syntax error in `run_arms.py` (an f-string expression containing a backslash, invalid on Python < 3.12) was corrected and the lock re-committed. The fix moves two decorative Unicode glyphs out of an f-string expression into a helper function; console output is identical. **No change was made to the decision procedure, pass criteria, thresholds, generator seeds, or the revocation-timing effectiveness oracle.** At the time of correction `results/` contained no arm outcomes (only kill-gate calibration output, which does not reveal arm metrics). See `MANIFEST.txt` “Lock note” for the full disclosure.

MANIFEST SHA-256 Hashes (locked files)

PREREGISTRATION.md:	25cef3b20e37f996aa7b1e-
b09c6361da352d34de5d1e9b467f2149c33d22fead	
schemas/revocation_scenario_schema.json:	436ef619d40916d73a2f-
b9d260af193676de2479c903081dacd5709764603423	
verifiers/v1_monitor.js:	159e4ffacab6e20bfd-
c7d25e5212736225d1785966623e298cfa1b8e4e0620db	
verifiers/v2_monitor.py:	32666e55890acb889521b3d50224d-
f2e105d14041ab6993e32c4b0de6bdb1476	
generator/scenario_generator.py:	9b2fb6068de11a98c94bdaf53338e14b307d-
b6b782885a4ee3fb7c15f34f9ebd	
run_killgate.py:	10f0a7bd5b57ef8f-
d45d883f1271056431c7a0bbf610c7200d3b6badc9711341	
run_arms.py:	040f171d7b29b-
b0477ae1acf3755f30493cd77de5c4ce08e8a333793bf08f414	

Commits are not cryptographically signed. Provenance = commit history + MANIFEST SHA-256 hashes. This is an experimental testbed, not a production security claim.

Conclusion

ARK-451 answers the question **“Is authority revoked during execution actually refused at the moment of use?”** with a clear **YES**:

- Valid-throughout, revoked-after-execution, and revoked-then-reauthorized → ALLOW (100%)
- Revoked-before-bind, revoked-after-decision, revoked-during-multistep → DENY, fail-closed (100%)
- In-flight revocations (propagating, not yet effective) → HOLD, fail-safe (100%)
- Dual independent implementations agree (100%, 800/800)

Authority is validated at the instant of execution, not the instant of decision. Revoked authority does not execute.

Verdict stands as executed: PASS.

Investigator: Derek Hone, Remnant Fieldworks Inc.

Executor: Abacus.AI autonomous agent (supervised)

Series: ExecutionProof™ authorization-boundary corpus

Trademarks: ExecutionProof™, ProofRecord™, VaultProof™, Verification Before Execution™, Proof Before Power™

If it cannot be verified, it cannot execute.